

COMP2123 Assignment 2

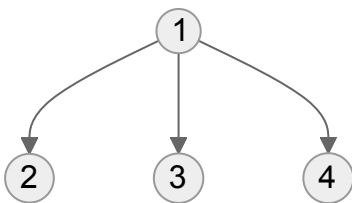
SID : 550686527

Problem 1

The correctness argument is not sound. The algorithm tests for a "principal edge", which is an edge in a tree of n nodes that splits the tree in two equal parts when removed. The primary issue in the correctness argument stems from the statement:

"Every time we make a recursive call, the edge splits the tree more evenly."

While this is true for even and odd binary trees, however it does not hold for cases of non binary trees, such as a star tree:



When the algorithm encounters this tree, starting from some arbitrary edge, it will first check if it the current edge is primary. If for example, we start at edge $(1, 2)$ - which is not principle as if removed, the remaining two trees $T_1 \{2\}$, $T_2 \{1, 3, 4\}$ are not less than or equal to $\lceil \frac{n}{2} \rceil$ - then the algorithm will more to an adjacent edge, minimising the maximum of the two subtrees created when that edge is removed. Since all edges will result in the same size of subtrees when removed, the algorithm will end up looping through the tree infinitely, never returning a principal edge as there is no principal edge. Furthermore, this occasion does not "split the tree more evenly", as it splits the tree the same amount each edge selection - therefore this statement of the correctness argument is disproved.

Another issue comes from the section of the correctness argument which claims:

"Eventually, the split is perfect."

Considering the case of trees with an odd number of nodes, the tree can never be split perfectly - you will always have a side with 1 more node than the other. These trees have two "principal edges", and no matter which the algorithm picks, the result is always a lopsided tree.

Problem 2

a) Algorithm

We are given a simple, connected, undirected graph $G = (V, E)$ in adjacency-list representation, and a vertex $u \in V$. We want to check whether u is a critical point, which occurs when there are no two vertices (v, w) adjacent to u such that there is a path in G from v to w not containing u .

Starting at u , the algorithm checks its adjacency list and marks all neighbours of u . It will then traverse to those neighbour lists of u , and temporarily remove any instance of u found

in these lists. We then run Breadth Search First (BSF)/Depth Search First (DSF) search algorithms on each of the separate neighbour lists. If we hit another neighbour in our BSF/DSF search, we can immediately terminate the algorithm and return that the point u is not critical, as you can travel between 2 neighbours without using the node u . If, in each instance of our BSF/DSF, we only ever count one neighbour (the neighbour node who's list it is), we can conclude and return that u is a critical point, as there is no way to get from one neighbour to another without adding u back into the neighbours adjacency list.

In steps, it would look something like this:

1. Search through target node u 's adjacency list
2. Mark all neighbours of u , and go to their lists to remove all instances of the node u
3. Perform DSF/BSF on all of these augmented neighbour lists
4. If you find ANOTHER neighbour (other than the one who's list it is) in your DSF/BSF, then u is not critical
5. If the only neighbour found is the one who's list it is - for all neighbour lists - then u is a critical node

b) Correctness

First, suppose the algorithm returns false - u is not a critical point. This only happens when the DSF/BSF finds TWO (it would terminate before it found more) distinct neighbours in the same adjacency list graph with u removed. This must mean that without u , you can still find a way from one neighbouring node v to another w , that only uses paths from a graph with u removed $G - u$. By definition of a critical point, u is not a critical point, therefore whenever the algorithm returns false, it is correct.

Now suppose the algorithm returns true. This means that every connected component of the graph $G - u$ contains at MOST one neighbour of u , and there is no way to travel between unconnected components, and thus neighbours of u . Assume for contradiction that u is not critical. Then there exists two distinct nodes v and w , both adjacent to u , such that there is a path from one to the other that does not contain u . But if such a path does not contain u , the path lies entirely inside a connected component of $G - u$. Therefore v and w must be in the same connected component of $G - u$, and thus that component would contain two neighbours of u , so the algorithm would have returned false. This contradicts the assumption the the algorithm returned true, therefore whenever the algorithm returns true, u is critical.

c) Time Complexity

Following the algorithm, we first mark all neighbours of u , so traversing through the adjacency list of u takes $O(\deg(u))$ time. Since $\deg(u) \leq n - 1$ and $\deg(u) \leq m$, this is within $O(m + n)$. Next, the algorithm runs a DSF/BSF on all the neighbour adjacency lists of u , with u removed. Assuming in the scenario where u is critical, all neighbour adjacency lists are distinct and separate from each other in the graph $G - u$, so each DSF/BSF runs in a combined amount of $O(m + (n - 1)) = O(m + n)$. Assuming the scenario where u is not critical, we will have "overlapping" neighbour adjacency lists, where we could potentially count the nodes in a connected graph with two neighbours twice, starting from a different

neighbour each time. However, as the algorithm terminates the moment it encounters two distinct neighbours in BSF/DSF, we will never run a BSF/DSF on at least the last neighbour adjacency list, and therefore our runtime for the BSF/DSF will be less than $O(m + n)$. Combining this all together, our overall time complexity for this algorithm will be:

$$O(\deg(u)) + O(m + n) = O(m + n)$$

Therefore this algorithm will check whether a node u in a graph is a critical point in $O(m + n)$ time.